WHITE PAPER · LANGUAGE DESIGN · NEURAL COMPUTATION · V5

NeuralLISP: A Homoiconic Language with Native Neural Network Primitives

When Code and Networks Are the Same Thing

Jonathan D. Horvat · Independent Research · 2026 · Revision 5

CONTENTS

1. Abstract
2. Introduction — The Impedance Mismatch Problem
3. Why LISP: The Homoiconic Foundation
4. Core Primitives: The Language Specification
5. Macros as the Neural Architecture Engine
6. Self-Modification as an Emergent Property
7. Neuro-Symbolic Reasoning: `defsymbolic`
8. Implementation Roadmap
9. The Agent System — `defagent` in Practice
10. The Phase 4 Bridge — Neural Scheduling on Bare Metal
11. The Neural OS Vision
12. Biological Parallels: Neuroplasticity and Embodied Cognition

13. Related Work and Differentiation

14. Conclusion: The Completion of an Idea

15. References

16. Appendix A: A Self-Improving MNIST Classifier

17. Appendix B: Formal Grammar of NeuralLISP Core Forms

ABSTRACT

Modern neural network frameworks treat the network as an object and the programming language as a separate tool used to manipulate it. This separation is practical but fundamentally misaligned: neural networks are graph-structured, self-adjusting computational artifacts, and the languages used to build them are not. The result is a persistent impedance mismatch — networks that cannot be directly inspected as programs, architectures that cannot be composed with the same tools used to compose code, and training processes that are structurally isolated from the languages that orchestrate them.

This paper specifies NeuralLISP, a Common Lisp-family language in which neural networks are not a library, not a framework, and not an abstraction layer. They are functions. A network is defined with `defnet` the way a function is defined with `defun` — it takes arguments, returns values, and composes with other functions. The difference is that `defnet` produces a differentiable function: one whose parameters are tracked by the runtime and whose gradient can be computed with respect to any loss.

NeuralLISP defines seven primitive forms: `defactivation`, `dense`, `conv`, `recur`, `attend`, `defnet`, and `with-training`. Above them, `defmodule` provides reusable parameterized subnetworks, `defagent` extends naturally to reinforcement learning, and `defsymbolic` bridges neural and symbolic reasoning — encoding logical constraints, domain rules, and formal fallacies as differentiable guards on network behavior. Because NeuralLISP is homoiconic — code and data share the same representation — a

running network can inspect and rewrite its own architecture using the same operations used to manipulate any other program. Self-modification is not a designed feature. It is an emergent consequence of treating networks as code in a language where code is data. [2]

```
(defnet classifier (x)
  (-> x
      (dense 256 relu :drop 0.2)
      (dense 128 relu :drop 0.2)
      (output 10 softmax)))
```

Changes in Revision 5

• New `defsymbolic` system: soft rules (loss penalties) and hard rules (rewrite guards) for neuro-symbolic reasoning.
• Logical fallacy registry: ~60 formal and informal fallacies as composable symbolic guards with auto-discovery.
• Neural OS vision: self-healing kernel, neural device drivers, architectural provenance as OS flight recorder.
• Post-numeric paradigm: weights as logic pointers, toward the "transistor moment" for AI.
• Biological parallels: structural neuroplasticity, Global Workspace Theory, introspection via the autograd tape.
• Safety and stability: `with-rewrite-sandbox`, approval hooks, rate limiting for self-modification.
• Lisp's historical role in AI and detailed JAX metaprogramming comparison expanded.

All code examples in this paper are conceptual — proposed syntax illustrating the language design, not listings from a running implementation. NeuralLISP is a specification and an argument for a direction, not a shipped system.

SECTION 1

Introduction: The Impedance Mismatch Problem

Neural networks are functions. Not metaphorically — structurally. A feedforward network takes an input tensor, passes it through a sequence of parameterized transformations, and returns an output tensor. That is what a function does. Recurrent networks are more precisely stateful functions — functions from input and accumulated hidden state to output and new hidden state — but the functional character is preserved; `recur` forms handle this state threading internally, keeping the programmer-facing model uniform.

LISP is also, fundamentally, about functions. Not just in the sense that it supports functional programming — in the deeper sense that functions are the organizing principle of the language. Functions are values. They can be passed as arguments, returned from other functions, stored in variables, and composed into larger functions. The entire LISP standard library is built on the idea that the most powerful thing you can do with a computation is hand it to another computation as an argument.[2]

These two things — neural networks and LISP — are describing the same underlying structure from different directions. A neural network is a composed function built from parameterized transformations. LISP is a language designed to construct, manipulate, and evaluate composed functions. The alignment is not superficial. It goes all the way down to what both things fundamentally are.

And yet the tools we use today to build neural networks treat the language and the network as entirely separate concerns. You write Python. Separately, a framework constructs a computation graph behind the scenes. The network is an object your code talks to, not something your code *is*. You cannot pass a trained network to `map`. You cannot partially apply a layer. You cannot compose two networks with the same syntax you use to compose two functions, because networks and functions are different categories of thing in every framework that exists today.

NeuralLISP eliminates this distinction by refusing to make it in the first place. In NeuralLISP, a network is defined the way a function is defined:

```
(defnet classifier (x)
  (-> x
      (dense  256 relu  :drop 0.2)
      (dense  128 relu  :drop 0.2)
      (dense  64  relu)
      (output 10  softmax)))
```

`classifier` is a callable. You invoke it with `(classifier input-tensor)`. You pass it to other functions. You compose it with other networks. The only difference between `defnet` and `defun` is that `defnet` produces a differentiable function — one whose parameters are tracked by the runtime and whose gradient flows back through every layer, through every composed activation, automatically.

The activation functions — `relu`, `softmax` — are not keywords dispatched on internally by the framework. They are symbols bound to functions in the environment, defined with `defactivation`, carrying both a forward computation and a backward gradient. They can be composed:

```
(dense 256 (compose batch-norm relu))
(dense 256 (partial leaky-relu 0.2))
(dense 256 (lambda (x) (* x (tanh (log (+ 1 (exp x))))))  ; mish
```

This design has three consequences developed in full throughout this paper. First, networks compose with the same syntax used to compose anything else in LISP. Second, the space of expressible activations is unbounded by what the language designers anticipated — any differentiable function is valid. Third, because NeuralLISP is homoiconic, a running network can reach into its own structure and rewrite it. Self-modification is not an API. It is what happens when you take homoiconicity seriously and apply it to something that matters.

Why LISP: The Homoiconic Foundation

In 1960, John McCarthy published a paper that introduced something unusual into the history of computing. He was not primarily trying to build a programming language. He was trying to find a notation for computation clean enough to reason about mathematically. What he arrived at, almost as a side effect, was a language in which programs and the data those programs operate on have the same structure — S-expressions. The property that falls out of this decision is called homoiconicity: code and data are the same kind of thing.[2]

McCarthy's paper established three things that remain the foundation of NeuralLISP's design. First, that a small number of primitive operations — `car`, `cdr`, `cons`, `eq`, `atom`, `cond`, and function application — are sufficient to build any computation expressible in the language. Second, that functions are data: a function can be represented as a list, stored, passed as an argument, and evaluated. Third, that the evaluator itself can be written in LISP, and is short enough to fit on a page. These three properties together mean that a LISP program can treat any other LISP program — including itself — as data to be read, modified, and re-evaluated.

This is not a technical curiosity. It is the most consequential design decision ever made in a programming language. Sheard and Peyton Jones's work on Template Haskell demonstrated that a type-safe macro system could be integrated into a compiled language, enabling compile-time program generation with strong correctness guarantees.[14] NeuralLISP's Phase 1 shape inference is analogous — static analysis made possible by the fact that network definitions are inspectable at read time. But Common Lisp's macro system, codified by Steele,[1] goes further: it operates on arbitrary list structure rather than a restricted grammar, giving NeuralLISP's architecture macros the ability to generate, validate, and transform network definitions as freely as any other list manipulation.

Lisp was conceived explicitly for artificial intelligence. At the 1956 Dartmouth conference where John McCarthy coined the term “artificial intelligence,” McCarthy sought a notation suitable for symbolic computation and mathematical reasoning about programs. By 1960, Lisp emerged — a language in which programs and data share the same list structure (S-expressions). This homoiconicity allows a program to treat other programs (or itself) as data.

Early AI was symbolic. Lisp excelled because it made recursion, first-class functions, and self-modification natural. Landmark systems such as SHRDLU (natural language understanding in a blocks world) and MACSYMA (symbolic algebra) were built in Lisp dialects. In the 1980s, specialized Lisp Machines from Symbolics, LMI, and Xerox ran entire operating systems and applications written in Lisp — the operating system was written in the same language as the applications, and every component was inspectable and modifiable at runtime. Lisp was, for a time, the de facto language of AI research.

The rise of statistical machine learning later shifted focus to numeric computation on commodity hardware. Yet the underlying alignment never disappeared: neural networks are composed differentiable functions — precisely the kind of structure Lisp was designed to manipulate, compose, and reason about.

NeuralLISP proposes to revive this original alignment. Networks would not be opaque objects; they would be modifiable programs in the language itself. The self-modifying MNIST classifier described in Appendix A, the neural scheduler proposed for a bare-metal hypervisor, the neuro-symbolic guards of `defsymbolic`, and the DXF round-trip are all direct descendants of McCarthy’s 1960 vision — a language that can think about itself, applied to a domain where thinking about yourself is the whole point.

WHAT HOMOICONICITY ACTUALLY MEANS

When the programmer writes a `defnet` form, that form is a list. The first element is the symbol `defnet`. The network’s topology — every layer, every activation, every keyword — is nested list structure. And every tool that operates on LISP lists operates

on that structure. The macro expander can transform it before evaluation. The reader can validate it at parse time. The evaluator can re-evaluate a modified version at runtime. None of this requires special machinery. It comes for free from the decision to write networks in LISP.

FUNCTIONS AS THE ORGANIZING PRINCIPLE

In Common Lisp, a function is a first-class value. You bind it with `defun`, store it with `#'`, pass it to `mapcar`, return it from another function with `lambda`, and compose it using standard higher-order operations. NeuralLISP builds on this directly. A layer form like `(dense 256 relu)` evaluates to a differentiable function. A `defnet` form evaluates to a differentiable function. The `->` threading macro composes these functions into a pipeline. The pipeline is a function. All the way up and all the way down, the organizing principle is the same.

The `->` threading macro follows thread-first semantics — the value threads as the first argument of each subsequent form — which is the correct choice since layer forms take their input tensor as the first argument. This convention is established in Clojure and widely understood; NeuralLISP adopts it directly.[13] Both `->` chains and regular function calls mix freely inside a `defnet` body, since the body is ordinary LISP.

THE MACRO SYSTEM AS ARCHITECTURE ENGINE

LISP macros operate at read time, before evaluation. A macro receives a list and returns a different list — the evaluator never sees the macro, only its result. This is the correct mechanism for expressing neural architecture families for two reasons. First, a macro can generate arbitrarily large `defnet` forms from a compact declaration. Second, a macro can validate structure before the network is built — catching shape mismatches at read time, before a single weight is initialized. NeuralLISP distinguishes between macro keyword arguments, which configure the macro's generation behavior, and primitive positional arguments, which appear in the expanded forms.

WHY COMMON LISP SPECIFICALLY

NeuralLISP is built in the tradition of Common Lisp and AutoLISP rather than Scheme or Clojure for practical reasons. Common Lisp has `defmacro`, a full macro system operating on arbitrary list structure. It has a condition system — a structured mechanism for signaling and handling exceptional situations more powerful than exception handling in any other mainstream language. `stop-training` is a condition, not a flag. `shape-mismatch` is a condition, not an exception. And SBCL compiles Common Lisp to native machine code with type declarations that achieve near-C performance on tight numerical loops — making a pure CL tensor backend viable without any foreign function interface. AutoLISP demonstrated that a Common Lisp-adjacent dialect embedded inside a complex domain-specific host could be used productively by domain experts who had no interest in becoming language theorists.[5] NeuralLISP applies the same philosophy to machine learning.

THE EVALUATOR AS A LEARNING LOOP

The LISP read-eval-print loop takes an expression, evaluates it, returns a result, and waits for the next expression. Neural network training takes a batch, runs a forward pass, computes a loss, runs a backward pass, updates weights, and waits for the next batch. These are structurally the same loop. In NeuralLISP, `with-training` is the REPL extended with gradient semantics. The language that defines the network is the same language that trains it, modifies it, and reasons about it. There is one environment, one evaluator, one loop.

SECTION 3

Core Primitives: The Language Specification

NeuralLISP has seven primitive forms from which every neural network can be expressed: `defactivation`, `dense`, `conv`, `recur`, `attend`, `defnet`, and `with-training`. Everything else — every architecture macro, every utility, every standard library network — is built from these seven things using the macro system and the standard LISP toolkit.

The rule governing argument style is simple and consistent throughout: if a parameter is always present and carries the primary meaning of the form, it is positional. If it is optional, an override, or a behavioral annotation, it is a keyword. No primitive form has more than three positional arguments. Every keyword argument is genuinely optional with a sensible default.

Companion forms. `pool`, `flatten`, and `norm` are derived companion forms, not primitives. They appear frequently alongside `conv` and `dense` but do not constitute independent primitives — each is a thin wrapper over tensor operations expressible in terms of the existing forms. The `output` form is a required terminal form within `defnet` bodies, not a standalone primitive; it declares the network's output width and final activation. `defweights` enables explicit weight sharing across layers — binding named weight tensors for use by multiple layer forms, essential for siamese networks and shared embeddings — following the same pattern as `defactivation`. `defloss` defines custom loss functions on the same model as `defactivation`.

Tensors as first-class values. A *tensor* in NeuralLISP is a first-class type — an `nl-tensor` struct backed by a row-major `(simple-array single-float (*))`, with shape metadata, gradient storage, and autograd tape linkage. Standard arithmetic — `tensor-add`, `tensor-sub`, `tensor-mul`, `tensor-div` — operates element-wise on the flat data vector. Reduction functions — `tensor-mean`, `tensor-sum`, `tensor-max` — accept tensors and optional axis arguments. Structural operations — `tensor-cat`, `tensor-reshape`, `tensor-transpose`, `tensor-flatten` — are standard library functions. This is why the residual connection `(+ x h)` in `defmodule`, the ensemble `(mean (mapcar ...))` in `defnet`, and the U-Net skip connections `(cat e4 e3)` in the diffusion macro are ordinary expressions rather than architectural primitives: tensors are values, arithmetic is defined on them, and ordinary function calls compose them. Tensor construction: `(torch-randn shape)`, `(torch-zeros shape)`, `(torch-ones shape)`. Weight initialization: `(xavier-uniform shape)`, `(kaiming-uniform shape)`. No CFFI, no foreign pointers — the tensor is a Common Lisp struct, inspectable with `describe`, serializable with `write`, and garbage-collected normally.

3.1 DEFACTIVATION — DIFFERENTIABLE FUNCTIONS AS VALUES

Before layer forms can be specified, activation functions must be. They are not keywords in NeuralLISP. They are values — callable objects carrying both a forward computation and a backward gradient, registered in the environment like any other symbol.

```lisp
(defactivation relu
  :forward  (lambda (x) (max 0.0 x))
  :backward (lambda (x) (if (> x 0.0) 1.0 0.0)))

(defactivation gelu
  :forward  (lambda (x) (* x (phi x)))
  :backward (lambda (x) (+ (phi x) (* x (gauss x)))))

(defactivation leaky-relu
  :forward  (lambda (slope x) (if (> x 0) x (* slope x)))
  :backward (lambda (slope x) (if (> x 0) 1.0 slope)))

; Standard library ships with: relu, sigmoid, tanh, gelu,
; softmax, linear, leaky-relu (default slope 0.2),
; swish, mish, elu, selu — all as named symbols.
```

After `defactivation`, `relu` is a symbol in the environment bound to a callable. Because activations are functions, they compose with the tools LISP already provides:

```lisp
; Composition — batch-norm then relu
(dense 256 (compose relu batch-norm))

; Partial application — leaky relu with fixed slope
(dense 256 (partial leaky-relu 0.15))

; Lambda inline — mish activation, no defactivation required
(dense 256 (lambda (x) (* x (tanh (log (+ 1.0 (exp x)))))))

; Shared activation bound to a variable
(defvar my-act (partial leaky-relu 0.15))
```

```
(defnet model (x)
  (-> x
      (dense 256 my-act)
      (dense 128 my-act)   ; same function object, not a copy
      (output 10  softmax)))
```

`autodiff` — the complement to `defactivation`. `defactivation` requires explicit `:forward` and `:backward` definitions — the programmer specifies both the function and its derivative. `autodiff` is the escape hatch when you want the gradient computed automatically from an arbitrary tensor expression rather than specified by hand. It wraps any lambda in NeuralLISP's native tape-based reverse-mode AD, recording every operation to build the backward pass automatically:

```
; autodiff traces tensor operations and computes gradients automatically
(defvar l2-norm-act
  (autodiff (lambda (x)
              (/ x (+ (torch-norm x 2) 1e-8)))))

(dense 256 l2-norm-act)   ; gradient flows through — no :backward needed
```

The constraint is that every operation inside the lambda must use NeuralLISP's tensor-aware arithmetic — the `ad-add`, `ad-mul`, `ad-tanh`, and so on that record to the autograd tape — rather than raw Common Lisp arithmetic on scalar numbers. Within that constraint, `autodiff` makes the entire tensor standard library differentiable. `defactivation` is the right choice when you have a clean mathematical definition and want the gradient explicitly stated. `autodiff` is the right choice when you want to experiment freely and let the tape work out the derivative.

3.2 DENSE — FULLY CONNECTED LAYER

```
(dense width activation)
(dense width activation :drop rate)
(dense width activation :init strategy)
(dense width activation :weights W)    ; defweights symbol
(dense width activation :as name)
```

ARG	TYPE	DEFAULT	MEANING
`width`	integer (positional)	required	Output units; input width inferred from prior layer
`activation`	function (positional)	required	Any `defactivation`, `compose`, `partial`, or `lambda`
`:drop`	float 0–1	`0.0`	Dropout rate after activation
`:init`	symbol	`:xavier`	`:xavier` `:he` `:random` `:zero`
`:weights`	`defweights` symbol	derived	Explicit weight tensor — enables weight sharing
`:as`	symbol	none	Local name for `connect` and `inspect`

A `dense` form evaluates to a differentiable function from input tensor to output tensor — a closure over a weight matrix `W` and bias `b`, applying `activation(xWᵀ + b)`. Weight and bias tensors are registered with the autograd system at evaluation time.

```
; Skip connection — let* names intermediate tensors, + adds them
(defnet residual-block (x)
  (let* ((h (-> x
              (dense 256 relu :as \'hidden)
              (dense 256 relu))))
    (+ x h)))   ; x bypasses both layers — residual connection

; Branch and merge — two parallel paths, concatenated
```

```
(defnet two-path (x)
  (let* ((path-a (-> x (dense 128 relu)))
         (path-b (-> x (dense 128 gelu)))
         (merged (cat path-a path-b)))
    (-> merged (output 10 softmax))))
```

Transfer learning is expressed naturally through `defweights` and the `:init :frozen` keyword. `:frozen` marks a weight tensor as non-trainable — the optimizer skips its gradient update, preserving the pretrained values while allowing surrounding layers to adapt:

```
; Extract weights from a trained network
(defweights shared-w1 :from (inspect pretrained :layer 1 :show :weights))
(defweights shared-w2 :from (inspect pretrained :layer 2 :show :weights))

; Fine-tune on a new task — frozen layers preserve pretrained knowledge
(defnet fine-tuned (x)
  (-> x
      (dense 256 relu :weights shared-w1 :init :frozen)
      (dense 128 relu :weights shared-w2 :init :frozen)
      (output 5   softmax)))  ; new task head trains freely
```

3.3 CONV — CONVOLUTIONAL LAYER

```
(conv filters activation kernel)
(conv filters activation kernel :stride s :pad p)

; Companion forms — derived, not primitive
(pool (2 2) max)
(pool (2 2) avg)
```

```
(pool (1 1) global)
(flatten)

(defnet image-classifier (x)
  (-> x
      (conv  32  relu (3 3))
      (pool  (2 2) max)
      (conv  64  relu (3 3))
      (pool  (2 2) max)
      (conv  128 (compose relu batch-norm) (3 3) :drop 0.25)
      (pool  (2 2) max)
      (flatten)
      (dense 512 relu :drop 0.25)
      (output 10  softmax)))
```

3.4 RECUR — RECURRENT LAYER

Cell type — `lstm`, `gru`, `rnn` — is positional because it fundamentally determines the layer's computational structure, parameter count, and gradient behavior.[7]

```
(recur width cell)
(recur width cell :bidir t :drop rate)

; Cell type as a runtime value
(defvar my-cell (if use-lstm \'lstm \'gru))
(recur 256 my-cell)

; Stacked bidirectional LSTM
(defnet sequence-model (x)
  (-> x
      (recur 256 lstm :bidir t :drop 0.3)
```

```
    (recur 256 lstm :bidir t :drop 0.3)
    (output 10 softmax)))
```

3.5 ATTEND — MULTI-HEAD ATTENTION LAYER

Heads and dim are both positional — together they determine key/query/value projection dimensions and parameter count. The reader validates that `heads` divides `dim` evenly at parse time.[3]

```
(attend heads dim)
(attend heads dim :drop rate :mask type)

(attend 8 512)
(attend 8 512 :drop 0.1 :mask :causal)  ; autoregressive
```

3.6 DEFNET — A DIFFERENTIABLE FUNCTION

`defnet` is specified deliberately to resemble `defun` as closely as possible. The body is almost always a `->` threading form describing the forward computation. `->` follows thread-first semantics: `(-> x f g h)` expands to `(h (g (f x)))`. Its definition is four lines of standard Common Lisp:

```
(defmacro -> (val &rest forms)
  "Thread val as the first argument through each form left-to-right.
   (-> x f g h) expands to (h (g (f x)))"
  (if (null forms) val
      `(-> ,(if (listp (car forms))
                `(,(caar forms) ,val ,@(cdar forms))
                `(,(car forms) ,val))
           ,@(cdr forms))))
```

```
; Single input — sequential
(defnet classifier (x)
  (-> x
      (dense  256 relu :drop 0.2)
      (dense  128 relu :drop 0.2)
      (output 10  softmax)))

; Multiple return values — VAE encoder
(defnet vae-encoder (x)
  (let* ((h       (-> x (dense 256 relu) (dense 128 relu)))
         (mu      (-> h (dense 32 linear :as \'mu-proj)))
         (log-var (-> h (dense 32 linear :as \'logvar-proj))))
    (values mu log-var)))

; Networks compose like functions — because they are functions
(defnet ensemble (x)
  (mean (mapcar (lambda (m) (funcall m x))
                (list model-a model-b model-c))))
```

Unit testing networks follows directly from their function nature:

```
; Shape contract test
(assert (equal (tensor-shape (encoder (torch-randn \'(1 784))))
               \'(1 32)))
```

PRIMITIVE	WHAT IT IS	EVALUATES TO
`defactivation`	Differentiable scalar function	Callable value in the environment
`dense`	Fully connected layer	Differentiable function

PRIMITIVE	WHAT IT IS	EVALUATES TO
`conv`	Convolutional layer	Differentiable function
`recur`	Recurrent layer	Differentiable function
`attend`	Multi-head attention layer	Differentiable function
`defnet`	Differentiable network function	Callable bound to a name
`with-training`	Training loop macro	Side effects + trained network

3.7 WITH-TRAINING — THE TRAINING MACRO

`with-training` must be a macro because its body — code evaluated each epoch — cannot be eagerly evaluated at call time. `stop-training` is a Common Lisp condition, not a flag. Loss functions are defined with `defloss` following the same pattern as `defactivation`.

Checkpointing saves both the network's definition list and its weight tensors. The definition is written as readable LISP source; the weights are a binary tensor file alongside it. Loading restores both: the definition is `eval`-ed to reconstruct the network symbol, and the weights are loaded into its parameter tensors.

```
; Checkpoint saves definition + weights
(checkpoint classifier "/models/run-42")

; Load restores the live network
(load-checkpoint "/models/run-42")
```

```
(with-training (net    classifier)
               (data   train-set :batch 64 :shuffle t)
               (loss   cross-entropy)
               (opt    (adam 0.001 :decay 1e-4))
               (epochs 100)
```

```
  (when (zero-mod epoch 10)
    (checkpoint classifier (format nil "/models/epoch-~d" epoch)))
  (when (< current-loss 0.01)
    (signal \'stop-training))
  ; Handle NaN loss
  (handler-case
      (when (nan-p current-loss)
        (load-checkpoint (format nil "/models/epoch-~d"
                                 (- epoch 10)))
        (halve-learning-rate opt))
    (nan-loss (c)
      (signal \'stop-training))))
```

```
; Optimizers — constructor functions, composable with schedule
(adam 0.001)
(sgd  0.01 :momentum 0.9 :nesterov t)
(schedule (adam 0.001) :warmup 1000 :decay cosine)

; Custom loss — defloss follows defactivation pattern
(defloss focal-loss
  (lambda (preds targets)
    (let ((ce (cross-entropy preds targets)))
      (* (expt (- 1 preds) 2) ce))))
```

Above the primitives. `defmodule` and `defagent`, specified in Section 4, are derived forms — each implementable entirely in terms of the seven primitives above and the standard LISP toolkit. `defmodule` expands to a `defun` augmented with `with-fresh-parameters`. `defagent` expands to two `defnet` forms, a replay buffer, and a `with-training` invocation. Neither requires evaluator changes or new primitives — they are macros, following the same extension mechanism available to any community package.

SECTION 4

Macros as the Neural Architecture Engine

The seven primitives are the assembly language of NeuralLISP. The macro system is the compiler. An architecture macro takes a compact declaration of intent and expands it into a fully specified collection of `defnet` forms at read time. The evaluator never sees the macro — only its result.

In every framework that exists today, high-level architecture abstractions are classes defined by framework authors and sealed by API boundaries. To customize them you subclass and override. In NeuralLISP, the high-level abstractions are macros. Their expansions are readable. `macroexpand` gives you the full expanded form and the expansion is just more NeuralLISP.

4.1 HOW ARCHITECTURE MACROS WORK

```
(defmacro dense-tower (name sizes activation output-width)
  `(defnet ,name (x)
     (-> x
         ,@(mapcar (lambda (w) `(dense ,w ,activation))
                   sizes)
         (output ,output-width softmax))))

; Used:
(dense-tower classifier (256 128 64) relu 10)

; Expands to:
(defnet classifier (x)
  (-> x
      (dense 256 relu)
      (dense 128 relu)
```

```
      (dense 64  relu)
      (output 10 softmax)))
```

4.2 THE STANDARD ARCHITECTURE MACROS

Feedforward / MLP — `defnet-mlp`

```
(defnet-mlp classifier
  :input 784  :hidden (256 128 64)
  :output 10  :act relu  :drop 0.2)
```

Convolutional Network — `defnet-cnn`

```
(defnet-cnn image-classifier
  :input-shape (1 28 28)
  :conv-stages ((32 (3 3)) (64 (3 3)) (128 (3 3)))
  :pool max  :dense (512 256)  :output 10  :drop 0.25)
```

Transformer — `defnet-transformer`[3]

```
(defnet-transformer language-model
  :vocab 50000  :dim 512  :heads 8
  :ff-dim 2048  :layers 6  :seq-len 1024  :drop 0.1)

; Each encoder block — generated 6 times:
(defnet language-model-block-N (x)
  (let* ((attn-out (-> x (norm 512) (attend 8 512 :drop 0.1 :mask :causal)))
         (x+attn   (+ x attn-out))              ; residual — tensor addition
         (ff-out   (-> x+attn (norm 512)
```

```
                                (dense 2048 gelu :drop 0.1)
                                (dense 512  linear)))
          (x+ff     (+ x+attn ff-out)))          ; residual — tensor addition
     x+ff))
```

Autoencoder — `defnet-autoencoder`

```
(defnet-autoencoder image-ae
  :input 784  :encoding (256 128)
  :latent 32  :decoding (128 256)  :output 784)
```

GAN — `defnet-gan`[4]

```
(defnet-gan image-gan
  :latent 100
  :generator     ((256 relu) (512 relu) (784 tanh))
  :discriminator ((512 leaky-relu) (256 leaky-relu) (1 sigmoid)))
```

The `adversarial` combinator is the one place where the standard `defnet` / `with-training` model is insufficient. A GAN is two networks trained in opposition. `adversarial` is a special form that registers this coupling — the acknowledged exception to the paper's claim that all architecture patterns are expressible through function composition.

Diffusion Model — `defnet-diffusion`

```
(defnet-diffusion image-diffusion
  :shape (3 64 64)  :steps 1000  :schedule cosine
```

```
  :features (64 128 256 512)  :attend-at (2 3))


; U-Net backbone — skip connections via let* and cat
(defnet image-diffusion (x t-embed)
  (let* ((e1  (-> x  (conv 64  relu (3 3))))
         (e2  (-> e1 (pool (2 2) max) (conv 128 relu (3 3))))
         (e3  (-> e2 (pool (2 2) max) (conv 256 relu (3 3))
                     (attend 8 256 :drop 0.1)))
         (e4  (-> e3 (pool (2 2) max) (conv 512 relu (3 3))
                     (attend 8 512 :drop 0.1)))
         (d3  (-> (cat e4 e3) (conv 256 relu (3 3))))
         (d2  (-> (cat d3 e2) (conv 128 relu (3 3))))
         (d1  (-> (cat d2 e1) (conv 64  relu (3 3)))))
    (conv 3 linear (1 1))))
```

4.3 DEFMODULE — REUSABLE PARAMETERIZED SUBNETWORKS

```
; Define a reusable residual block
(defmodule residual-block (x in-dim out-dim)
  (let ((h (-> x
               (norm  in-dim)
               (dense out-dim relu :drop 0.1)
               (norm  out-dim))))
    (+ x h)))   ; residual: tensor addition


; Instantiate multiple times with different dims
(defnet resnet-6 (x)
  (-> x
      (conv   64  relu (3 3))
      (residual-block x 64  64)
```

```
    (residual-block x 64  64)
    (residual-block x 64  128)
    (pool   (2 2) avg)
    (flatten)
    (output 10 softmax)))
```

	DEFNET	DEFMODULE
Arguments	Input tensors only	Input tensors + dimension parameters
Parameters resolved	At definition time	At instantiation time
Weight allocation	Once, at eval	Fresh per instantiation
Primary use	Top-level named networks	Reusable architectural patterns

4.4 REINFORCEMENT LEARNING — DEFAGENT

```
(defagent rl-agent
  :policy  (defnet policy-net (state)
             (-> state
                 (dense 256 relu)
                 (dense 128 relu)
                 (output n-actions softmax)))
  :value   (defnet value-net (state)
             (-> state
                 (dense 256 relu)
                 (dense 128 relu)
                 (output 1 linear)))
  :env     cart-pole
  :gamma   0.99
  :reward  (lambda (state action next-state done)
             (if done -1.0 1.0)))
```

`defagent` is the clearest demonstration that NeuralLISP's design generalizes beyond supervised learning without requiring new primitives. RL is not a special case. It is a natural consequence of having differentiable functions as values, training loops as macro bodies, and loss functions as callables.

4.5 MACROS ARE READABLE

Every macro can be fully expanded at the REPL with `macroexpand`. A macro package convention allows community-published architectures as ordinary Common Lisp packages: `(require :neurallisp-mamba)` makes `defnet-mamba` available. Packages are distributed as ASDF system definitions and installed via Quicklisp.

SECTION 5

Self-Modification as an Emergent Property

Nothing in NeuralLISP was designed specifically to enable self-modification. There is no self-modification API, no special runtime mode, no flag to enable it. It emerges from two decisions made for completely independent reasons: networks are functions, and functions are values.

When a network is a function and a function is a value, a running network can reach into its own definition, read it, and rewrite it — through the same operations that reach into and rewrite any other value in a LISP program.

5.1 WHAT A NETWORK DEFINITION IS

When the programmer writes a `defnet` form, `defnet` binds the symbol in the environment to a differentiable function and also stores the network's definition — the list structure of the form — as a property of that symbol. The definition and the function are two faces of the same object.

```
(inspect classifier)
; => (defnet classifier (x)
;      (-> x
;          (dense  256 relu  :drop 0.2)
;          (dense  128 relu  :drop 0.2)
;          (output 10  softmax)))

(inspect classifier :layer 2)
; => (dense 128 relu :drop 0.2)

(inspect classifier :layer 2 :pos 2)
; => relu     ; the activation — a symbol bound to a function
```

5.2 REWRITING A LIVE NETWORK

```
; Replace a layer entirely
(rewrite classifier :layer 2
  (dense 256 relu :drop 0.3))

; Change only the activation function
(rewrite classifier :layer 2 :pos 2 \'gelu)

; Insert before a position
(rewrite classifier :insert-before :layer 3
  (norm 128))

; Modification history
(history classifier)
; => ((t=0      :defined)
;     (epoch=12 :rewrite :layer 2 :from (dense 128 relu :drop 0.2)
```

```
;                                   :to   (dense 256 relu :drop 0.3))
;     ...)
```

5.3 ACTIVATION FUNCTIONS AS MODIFICATION TARGETS

```
; Targeted replacement
(rewrite classifier :layer 2 :pos 2 \'gelu)

; Universal replacement
(rewrite-all classifier :pos 2 \'gelu)

; Predicate replacement — only layers currently using relu
(rewrite-where classifier
  (lambda (layer) (eq (layer-activation layer) \'relu))
  :pos 2 \'gelu)
```

5.4 SELF-MODIFICATION INSIDE THE TRAINING LOOP

```
; Progressive depth growth
(with-training (net    growing-net)
               (data   train-set :batch 32)
               (loss   cross-entropy)
               (opt    (adam 0.001))
               (epochs 500)
  (when (and (zero-mod epoch 50)
             (> current-loss 0.4))
    (rewrite growing-net :insert-before :output
      (dense (* 2 (width-of growing-net :last-hidden))
```

```
              relu :drop 0.2)))
  (log-loss epoch))
```

```
; Activation mutation under vanishing gradients
(with-training (net    deep-net)
               (data   train-set :batch 64)
               (loss   cross-entropy)
               (opt    (adam 0.001))
               (epochs 300)
  (when (loss-stagnant-p :window 20 :delta 1e-4)
    (let* ((worst-idx    (min-gradient-layer deep-net))
           (current-act  (layer-activation deep-net worst-idx))
           (next-act     (activation-successor current-act)))
      (rewrite deep-net :layer worst-idx :pos 2 next-act)))
  (log-loss epoch))
```

5.5 NETWORKS THAT WRITE NETWORKS

Because network definitions are lists, a function that returns a list can return a `defnet` form, and `eval` evaluates any form in the current environment.

```
(defun embedding->defnet (embedding name)
  (let* ((widths  (extract-widths embedding))
         (acts    (extract-activations embedding))
         (layers  (mapcar (lambda (w a) `(dense ,w ,a))
                          widths acts)))
    `(defnet ,name (x)
       (-> x ,@layers (output 10 softmax)))))
```

```
; eval is the bridge — one function call
(eval (embedding->defnet controller-output \'generated-net))
```

```
(defagent architecture-agent
  :policy  (defnet arch-controller (task-embedding)
             (-> task-embedding
                 (dense 256 relu)
                 (dense 128 relu)
                 (output 32 linear)))  ; architecture embedding
  :value   (defnet arch-critic (task-embedding)
             (-> task-embedding
                 (dense 256 relu)
                 (output 1 linear)))
  :env     neurallisp-eval-env
  :reward  (lambda (embedding action result done)
             (validation-accuracy result)))
```

5.6 CONSTRAINTS AND STABILITY

Shape validation before commitment. Weight preservation by default. Lazy gradient graph reconstruction. Modification history as a first-class auditable value.

```
(preview-rewrite classifier :layer 2
  (dense 256 gelu :drop 0.3))
; =>
;   BEFORE: (dense 128 relu :drop 0.20)   params=32,896
;   AFTER:  (dense 256 gelu :drop 0.30)   params=65,792
;   shape contract: valid
```

```
;   layer 2: reinitialized (:xavier)
;   layers 1, 3, 4: weights preserved
```

5.7 SAFETY, STABILITY, AND THE REWRITE SANDBOX

```
(with-rewrite-sandbox (net classifier)
  (rewrite classifier :layer 2 :pos 2 \'gelu)
  (rewrite classifier :insert-before :output
    (dense 256 relu :drop 0.1))

  ;; Validate: run a test batch and check accuracy
  (:validate
    (let ((acc (evaluate classifier *val-set* :metric :accuracy)))
      (> acc (* 0.95 *baseline-accuracy*))))

  ;; Optional: require human approval for production systems
  (:approve :human)
  (:rate-limit 3 :per :epoch))
```

5.8 ARCHITECTURAL PROVENANCE: THE FLIGHT RECORDER

NeuralLISP's modification history is the flight recorder for neural architectures. Every `rewrite` — its timestamp, the before-state, the after-state, the triggering condition, and the epoch context — would be appended to an immutable log.

```
(history classifier)
;; => ((t=0      :defined)
;;     (epoch=12 :rewrite :layer 2 :from (dense 128 relu :drop 0.2)
;;                                 :to   (dense 256 relu :drop 0.3)
;;                                 :reason :loss-stagnant
```

```
;;                                    :approved-by :auto)
;;     (epoch=50 :sandbox-rejected :proposed (dense 512 gelu)
;;                                    :reason :accuracy-degraded)
;;     ...)

;; Replay to any point in time
(replay-history classifier :to-epoch 12)

;; Diff between any two points
(history-diff classifier :from-epoch 0 :to-epoch 50)
```

SECTION 6

Neuro-Symbolic Reasoning: `defsymbolic`

The neural-symbolic research tradition has long asked how statistical learning and logical reasoning can be combined. Most approaches treat the boundary between the two as a hard interface. NeuralLISP takes a different position: in a homoiconic language where networks are programs and programs are data, the boundary between symbolic reasoning and neural computation is not a bridge to be built. It is a distinction that dissolves.

`defsymbolic` is the form that makes this dissolution explicit. It defines logical constraints, domain rules, and formal reasoning patterns as composable guards on network behavior. These guards operate in two modes: **soft rules** that add penalty terms to the loss function during training, and **hard rules** that constrain the `rewrite` system, preventing structural modifications that would violate specified invariants.

6.1 SOFT RULES: LOSS PENALTIES FROM LOGIC

```
;; Define a symbolic constraint: output probabilities must sum to 1
(defsymbolic probability-constraint :soft
  :predicate (lambda (output)
               (approx-equal (tensor-sum output) 1.0 :tolerance 0.01))
  :penalty   (lambda (output)
               (tensor-square (- (tensor-sum output) 1.0)))
  :weight    0.1)

;; Domain-specific: medical diagnosis must not contradict known exclusions
(defsymbolic mutual-exclusion :soft
  :predicate (lambda (output)
               (not (and (> (tensor-ref output 3) 0.5)
                         (> (tensor-ref output 7) 0.5))))
  :penalty   (lambda (output)
               (* (tensor-ref output 3) (tensor-ref output 7)))
  :weight    1.0)

;; Attach constraints to a network
(attach-symbolic classifier
  \'(probability-constraint mutual-exclusion))
```

6.2 HARD RULES: REWRITE GUARDS

```
;; Hard rule: minimum network depth
(defsymbolic min-depth-guard :hard
  :invariant (lambda (net proposed-rewrite)
               (>= (layer-count-after net proposed-rewrite) 3))
  :message "Cannot reduce network below 3 layers")

;; Hard rule: output width must remain 10
```

```
(defsymbolic output-width-guard :hard
  :invariant (lambda (net proposed-rewrite)
               (= (output-width-after net proposed-rewrite) 10))
  :message "Output width must remain 10 for MNIST classification")

(attach-guard classifier
  \'(min-depth-guard output-width-guard))
```

6.3 THE LOGICAL FALLACY REGISTRY

NeuralLISP's fallacy registry encodes approximately sixty formal and informal fallacies as composable symbolic guards.[26]

```
;; Formal fallacy: affirming the consequent
(defsymbolic affirming-consequent :soft
  :predicate (lambda (net input output causal-graph)
               (not (consequent-drives-antecedent-p
                      output causal-graph)))
  :penalty   (lambda (net input output causal-graph)
               (consequent-antecedent-penalty output causal-graph))
  :weight    0.5
  :category  :formal)

;; Auto-discovery: attach all known fallacy guards at once
(attach-fallacy-guards reasoning-net :all)

;; Or selectively by category
(attach-fallacy-guards reasoning-net :formal :relevance :ambiguity)
```

6.4 THE POST-NUMERIC PARADIGM

Today, a trained weight is a floating-point number. Its meaning is implicit. In NeuralLISP, through `defsymbolic`, a weight's behavior would be constrained by logical rules. Through the modification history, every weight's lineage would be recorded. That weight is no longer "just a number." It is a logic pointer: a numeric value whose semantics are tracked by the symbolic system.

The transistor was not an improvement on the vacuum tube. It was a different kind of thing entirely. The post-numeric weight — a parameter whose meaning is tracked, constrained, and auditable — would not be an improvement on the floating-point weight. It would be the beginning of a different kind of neural computation.

SECTION 7

Implementation Roadmap

NeuralLISP is not a thought experiment requiring exotic technology. SBCL is production-grade software. The macro system is the same one that has shipped with Common Lisp implementations since the 1980s. The autograd engine is native Common Lisp — tape-based reverse-mode AD operating directly on the homoiconic representation.

7.1 HOST PLATFORM

SBCL is the primary host — it compiles to native machine code with type declarations that achieve near-C performance. **ECL** compiles Common Lisp to C for WebAssembly deployment. **Clasp** is built on LLVM. The only external dependency is `bordeaux-threads`. No CFFI, no LibTorch, no cmake. `(ql:quickload :neurallisp-pure)` and you are running.

7.2–7.4 PHASES 1–3

```lisp
;;; The tensor struct — the atom of NeuralLISP's computation
(defstruct nl-tensor
  (shape   nil :type list)                    ; e.g. (64 784)
  (data    nil :type (simple-array single-float (*)))
  (numel   0   :type fixnum)
  (requires-grad-p nil :type boolean)
  (grad    nil)                               ; nl-tensor or nil
  (tape-node nil))                            ; autograd linkage
```

```lisp
;;; The tape node — one per differentiable operation
(defstruct tape-node
  (output   nil :type nl-tensor)
  (inputs   nil :type list)
  (backward nil :type function))

;;; Backward pass — reverse-mode AD over the tape
(defun backward (loss)
  (setf (nl-tensor-grad loss) (ones-like loss))
  (dolist (node (reverse *tape*))
    (let* ((grad-out (nl-tensor-grad (tape-node-output node)))
           (grad-ins (funcall (tape-node-backward node) grad-out)))
      (loop for input in (tape-node-inputs node)
            for igrad in grad-ins
            when (and input igrad)
            do (%accumulate-grad input igrad))))
  (clear-tape))
```

7.5 PHASE 4 — THE INTERACTIVE REPL

```
NeuralLISP> classifier

  classifier
  ────────────────────────────────────────
  (dense  256  relu    drop=0.20)  params=200,960
  (dense  128  relu    drop=0.20)  params=32,896
  (output  10  softmax)            params=1,290
  ────────────────────────────────────────
  total params:  235,146
  last loss:     0.0842   (epoch 20)
  last accuracy: 97.3%    (val)
  modifications: 2
```

```
; Free a network — GC reclaims the weight arrays
(undefnet classifier)
(mapc #\'undefnet (list \'model-a \'model-b \'model-c))
```

7.7 VISUAL TOOLING: THE DXF ROUND-TRIP

A neural network architecture diagram is a visual representation of exactly the same information stored in a `defnet` form. NeuralLISP closes this gap by making the drawing and the program the same artifact via DXF. DXF supports `BLOCK` definitions and `ATTRIB` tags. A NeuralLISP layer is a `BLOCK` instance. Its arguments are `ATTRIB` tags on that instance. Connection polylines encode the topology. The entire `defnet` form is encoded in the drawing. The drawing is the program.

```
; A dense layer in NeuralLISP source
(dense 256 relu :drop 0.2 :as encoder-hidden)
```

```
; The same layer as a DXF BLOCK instance
; INSERT: NEURALLISP-LAYER block at position (100, 200)
;   ATTRIB TAG=TYPE    VALUE=DENSE
;   ATTRIB TAG=WIDTH   VALUE=256
;   ATTRIB TAG=ACT     VALUE=RELU
;   ATTRIB TAG=DROP    VALUE=0.2
;   ATTRIB TAG=AS      VALUE=ENCODER-HIDDEN
;   ATTRIB TAG=PARAMS  VALUE=200960   ; read-only, computed
```

The complete `neurallisp-visual` package — including all DXF section writers (`write-dxf-header`, `write-dxf-tables`, `write-dxf-blocks`, `write-layer-insert`, `write-connection-arrow`), the ATTRIB parser (`parse-dxf-attribs`), the layer form reconstructor (`rebuild-layer-form`), the full `visualize` function, `import-from-dxf`, `show-net`, `diff-nets`, and `compare-nets` — is specified in full in the HTML source of this paper. The implementation handles DXF group code/value pairs, proper `ATTDEF`/`ATTRIB` sequencing, `ENDBLK` placement, symbol interning into `:neurallisp`, and accurate parameter computation carrying input width through the layer chain.

```
; Export at the REPL
(visualize classifier)
; => Exported CLASSIFIER → classifier.dxf

; Engineer opens classifier.dxf in AutoCAD or LibreCAD.
; Double-clicks layer 2 block. Changes WIDTH 128→256, ACT RELU→GELU.
; Saves file. Returns to REPL.

(import-from-dxf "classifier.dxf" :into \'classifier)
; => Layer 2: width 128→256, activation relu→gelu (reinitialized)
;    All other layers: weights preserved
```

```
; Visual diff between two network versions
(compare-nets classifier-v1 classifier-v2 :file "comparison.dxf")
; Changed layers → red. Unchanged → default. Added → green.
```

PHASE	DELIVERABLE	STATUS
1	Reader, `defactivation`, shape inference, `validate-activation`	Proposed
2	Pure CL tensor backend, forward pass, `infer` works	Proposed
3	Tape-based autograd, `with-training`, optimizers, `rewrite` in training body	Proposed
3b	`defagent` actor-critic, replay buffer, message passing, neural scheduler	Proposed
3c	Phase 4 bridge: vCPU encoding, VM simulation, `schedule-next-vcpu-neural`	Proposed
4	REPL display, `preview-rewrite`, incremental training	Proposed
5	Architecture library, `defloss`, introspection utilities, packaging	Proposed
6	`neurallisp-visual`: DXF export, import, `compare-nets`, `diff-nets`	Proposed
7	`defsymbolic`: soft/hard rules, fallacy registry, rewrite guards	Proposed

SECTION 8

The Agent System — `defagent` in Practice

The agent system implements actor-critic reinforcement learning, thread-safe message passing, and neural process scheduling, all on the pure CL tensor backend with zero foreign dependencies.

8.1 ACTOR-CRITIC ARCHITECTURE

```
;;; The agent struct
(defstruct nl-agent
  (policy-sym  nil :type symbol)
  (value-sym   nil :type symbol)
  (replay-buf  nil :type replay-buffer)
  (gamma       0.99 :type single-float)
  (gae-lambda  0.95 :type single-float)
  (entropy-coeff 0.01)
  (value-coeff   0.5)
  (max-grad-norm 0.5)
  (episode-count 0)
  (total-steps   0)
  (reward-history nil :type list))

(defagent my-agent
  :obs-dim    23
  :act-dim    5
  :hidden     (64 32)
  :gamma      0.99
  :gae-lambda 0.95)

; Expands to:
(defnet my-agent-policy (x)
  (-> x (dense 64 relu) (dense 32 relu) (output 5 softmax)))

(defnet my-agent-value (x)
  (-> x (dense 64 relu) (dense 32 relu) (output 1 linear)))
```

```
;;; Mailbox — bounded queue with lock + condition variable
(defstruct mailbox
  (queue    nil :type list)
  (capacity 1024 :type fixnum)
  (lock     (bt:make-lock "mailbox"))
  (not-empty (bt:make-condition-variable)))

;;; Named multicast channels
(subscribe   \'sched-agent :system-state)
(publish     :system-state
  (make-state-msg :process-table procs))
(agent-receive \'sched-agent)
```

8.3 NEURAL PROCESS SCHEDULER

```
;;; State encoding: 4 processes × 5 features + 3 global = 23 dim
(encode-system-state process-table)

;;; One-liner: create and train a neural scheduler
(define-scheduler \'my-sched :max-procs 4 :hidden (64 32))
(train-scheduler  \'my-sched :episodes 1000)

;;; The trained scheduler is a function — drop-in replacement
(defvar sched-fn (make-neural-scheduler-fn \'my-sched))
(funcall sched-fn process-table)  ; → pid of process to run next
```

SECTION 9

The Phase 4 Bridge — Neural Scheduling on Bare Metal

The bridge maps Phase 4 Lisp OS's concrete vCPU structs into NeuralLISP's tensor representation, replacing the hand-coded round-robin scheduler with a learned policy while preserving a fallback path for safety.

```
(defun schedule-next-vcpu-neural (vm)
  "Neural scheduler with round-robin fallback."
  (let* ((state   (encode-vm-state vm))
         (action  (agent-act (neural-sched-state-agent-sym *neural-sched*)
                             state :greedy t))
         (vcpu-idx action))
    (if (vcpu-schedulable-p vm vcpu-idx)
        vcpu-idx
        (round-robin-fallback vm))))

;;; Three code changes integrate NeuralLISP into Phase 4:
;  1. smp.lisp:    schedule-next-vcpu calls schedule-next-vcpu-neural
;  2. exit-handler: handle-vm-exit calls (observe-vm-exit exit-reason)
;  3. exit-handler: handle-exit-hlt calls (update-vcpu-state idx :halted)
```

```
;;; Simulated VM for training — no VT-x hardware required
(train-phase4-scheduler
  :num-vcpus 4
  :episodes  500
  :hidden    (64 32)
  :gamma     0.99
  :reward-fn #\'%default-vcpu-reward)
```

The Neural OS Vision

An operating system in which every policy decision — scheduling, memory management, I/O priority, device configuration — is a `defagent` whose behavior adapts to observed workload patterns and whose modifications are governed by `defsymbolic` guards.

10.1 THE SELF-HEALING KERNEL

```
;; The memory policy agent detects degrading hardware
(with-training (net memory-policy-agent)
               (data (observe-ecc-events))
               (loss mse)
               (opt (adam 0.001))
               (epochs :continuous)
  (when (ecc-rate-exceeds-p :dimm 2 :threshold 0.001)
    (with-rewrite-sandbox (net memory-policy-agent)
      (rewrite memory-policy-agent
        :avoid-region (dimm-address-range 2))
      (:validate (memory-invariants-hold-p))
      (:approve :auto)
      (:log-reason "ECC degradation on DIMM 2"))))
```

10.2 NEURAL DEVICE DRIVERS

```
(defagent nic-coalescing-agent
  :policy  (defnet nic-policy (state)
             (-> state
                 (dense 32 relu)
                 (dense 16 relu)
```

```
                (output 4 softmax)))
  :value   (defnet nic-critic (state)
             (-> state
                 (dense 32 relu)
                 (output 1 linear)))
  :env     nic-environment
  :gamma   0.95
  :reward  (lambda (state action next-state done)
             (nic-throughput-latency-reward next-state)))
```

10.3 ARCHITECTURAL PROVENANCE AS OS TELEMETRY

```
(history scheduler-agent :since (- (get-universal-time) 3600))
(history-diff memory-agent :from-boot :to-now)
(correlate-histories scheduler-agent latency-monitor
  :window 300)
```

SECTION 11

Biological Parallels: Neuroplasticity and Embodied Cognition

11.1 STRUCTURAL NEUROPLASTICITY

The brain does not have a fixed architecture. Neurons form new connections (synaptogenesis), prune unused ones (synaptic pruning), and even grow new neurons in specific regions (neurogenesis in the hippocampus). NeuralLISP's `rewrite` is structural plasticity for artificial networks. When a training loop inserts a new layer, it is performing synaptogenesis. When it removes a layer whose gradients have collapsed, it is performing synaptic pruning.

11.2 GLOBAL WORKSPACE THEORY AND THE REPL

Baars's Global Workspace Theory (GWT) proposes that consciousness arises from a "global workspace" — a shared information space that makes selected contents available to all specialized cognitive processes simultaneously.[25] The NeuralLISP REPL is structurally analogous. The autograd tape is NeuralLISP's proposed introspection mechanism — the network's access to its own computational process.

11.3 EMBODIED COGNITION AND THE PHASE 4 BRIDGE

The Phase 4 bridge would be the closest artificial analogue to embodied cognition in the ML literature. The neural scheduler would operate on real hardware state — vCPU exit counts, IRQ pending flags, timer registers. Its decisions would have immediate physical consequences. The "body" is the hardware. The "environment" is the workload. The "sensorimotor loop" is the observe-decide-act cycle.

SECTION 12

Related Work and Differentiation

12.1 DIFFERENTIABLE PROGRAMMING AND JAX

JAX treats Python functions as transformable objects — `grad`, `jit`, `vmap`. NeuralLISP and JAX share the core intuition. They differ in how completely they follow it. JAX is a Python library: the language and the neural computation remain distinct layers. Python's syntax is fixed. Its macro system does not exist. The architecture macros of Section 4 are impossible in JAX because Python has no macro system.

12.6 WHAT NEURALLISP DOES THAT NONE OF THESE DO

Seven properties, not previously achieved simultaneously:

Networks as differentiable functions in a homoiconic language. The form you write is both the definition and the executable.

Activation functions as first-class composable values. Composable with `compose`, fixable with `partial`, writable with `lambda`.

Self-modification as an intrinsic property. A consequence of homoiconicity, not an API.

Neuro-symbolic reasoning as a language feature. `defsymbolic` bridges logical constraints and neural behavior.

A unified execution model. The REPL loop and the training loop are the same loop.

Homoiconicity extended into the visual domain via reversible DXF.

Zero foreign dependencies. The entire system is inspectable all the way down.

SECTION 13

Conclusion: The Completion of an Idea

In 1958, John McCarthy set out to find a notation for computation clean enough to reason about mathematically. What he found was more consequential than a notation. He found that if you build a language around a single commitment — that programs and data are the same kind of thing — the language acquires properties that no amount of subsequent feature addition can replicate.[2]

Neural networks are the dominant form of machine learning today. They are also the most natural domain to which McCarthy's original commitment should be applied. A neural network is a composed function built from parameterized transformations. LISP is a language organized around the composition of functions. A neural network learns by adjusting its own parameters in response to data. A LISP program can adjust its own structure in response to computation. NeuralLISP is the recognition of an alignment that existed before either thing had a name.

This paper specifies seven irreducible primitives and a clear account of how everything else is derived from them. From these seven primitives, standard library macros would derive every major neural network architecture. The entire design targets pure Common Lisp with zero foreign dependencies.

Self-modification would not need to be designed into NeuralLISP. It is what happens when you build a language in which networks are functions, functions are values, and values are lists. Neuro-symbolic reasoning would not need to be a separate feature. The fallacy registry would not need to be a knowledge base.

LISP gave us a language that could think about itself. NeuralLISP puts the neural network inside the language that was always designed to hold it.

The boundary between a program and its own description was supposed to dissolve. NeuralLISP is the dissolving.

✦ ✦ ✦

References

[1] Steele, G. L. *Common Lisp: The Language*, 2nd ed. Digital Press, 1990. The definitive specification of Common Lisp, including the `defmacro` system, condition system, and reader machinery that form NeuralLISP's host foundation.

[2] McCarthy, J. "Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I." *Communications of the ACM*, 3(4):184–195, 1960. The original paper establishing LISP, homoiconicity, and the equivalence of code and data as list structures.

[3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. "Attention Is All You Need." *Advances in Neural Information Processing Systems* 30 (NeurIPS 2017),

pp. 5998–6008.

[4] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. “Generative Adversarial Nets.” *Advances in Neural Information Processing Systems* 27 (NIPS 2014), pp. 2672–2680.

[5] Autodesk, Inc. *AutoLISP Developer’s Guide*. Autodesk, Inc., various editions. Operational specification of AutoLISP as embedded in AutoCAD; primary model for NeuralLISP’s domain-embedded LISP design philosophy.

[6] He, K., Zhang, X., Ren, S., and Sun, J. “Deep Residual Learning for Image Recognition.” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[7] Hochreiter, S. and Schmidhuber, J. “Long Short-Term Memory.” *Neural Computation*, 9(8):1735–1780, 1997.

[8] LeCun, Y., Bengio, Y., and Hinton, G. “Deep Learning.” *Nature*, 521:436–444, 2015.

[9] Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. “Automatic Differentiation in Machine Learning: A Survey.” *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[10] Garcez, A. d’A., Lamb, L. C., and Gabbay, D. M. *Neural-Symbolic Cognitive Reasoning*. Springer, 2009.

[11] Chollet, F. *Deep Learning with Python*. Manning Publications, 2017.

[12] Paszke, A., Gross, S., Massa, F., et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Advances in Neural Information Processing Systems* 32 (NeurIPS 2019).

[13] Hickey, R. *Clojure Reference: Threading Macros*. Rich Hickey / Clojure Core, 2007–present.

[14] Sheard, T. and Peyton Jones, S. “Template Meta-programming for Haskell.” *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell*, pp. 1–16, 2002.

[15] Melis, G. *MGL: A Common Lisp Machine Learning Library*. https://github.com/melisgl/mgl (ongoing).

[16] Holzmann, M. “LispNet: A machine learning framework for Petalisp…” Master’s thesis, FAU Erlangen-Nürnberg, 2022.

[17] Borodust. *claw-torch: Common Lisp bindings to LibTorch*. https://github.com/borodust/claw-torch.

[18] Pearlmutter, B. A. & Siskind, J. M. “Reverse-mode AD in a functional framework.” *ACM TOPLAS*, 2008.

[19] Horvat, J. D. “Intelligence at the Edge: A Two-Model Architecture for Personal AI.” Independent Research, 2026.

[20] Horvat, J. D. “Intelligence per Milliwatt: Replacing Tokens-per-Second as the Primary Metric for Edge AI.” Independent Research, 2026.

[21] Horvat, J. D. “HAZOP for AI Systems: Applying Process Safety Methodology to Artificial Intelligence.” Independent Research, 2026.

[22] Horvat, J. D. “A P&ID Notation Standard for AI System Architectures.” Independent Research, 2026.

[23] Horvat, J. D. “Behavioral Observation as a Training Signal for Personal AI Adapters.” Independent Research, 2026.

[24] Horvat, J. D. “Why Broadcast Synchronization Is Not Federated Learning.” Independent Research, 2026.

[25] Baars, B. J. *A Cognitive Theory of Consciousness*. Cambridge University Press, 1988.

[26] Aristotle. *Sophistical Refutations* (trans. W. A. Pickard-Cambridge). Clarendon Press, c. 350 BCE / 1928.

APPENDIX A

A Self-Improving MNIST Classifier

The following is a conceptual NeuralLISP program illustrating the full design in action. It loads MNIST, trains a minimal two-layer classifier, and progressively deepens its own architecture in response to training dynamics.

```
;;;; self-improving-mnist.lisp
;;;; Starts as a 2-layer MLP. Ends as whatever the training signal demands.

(require :neurallisp)
(in-package :neurallisp)

;; Data
(defvar *train* (load-dataset :mnist :split :train :normalize t))
(defvar *val*   (load-dataset :mnist :split :val   :normalize t))
```

```
;; Initial architecture — minimal, intentionally underpowered
(defnet mnist-net (x)
  (-> x
      (dense  64  relu)
      (output 10  softmax)))

;; Activation ladder — ordered by gradient quality
;; relu → gelu → swish → mish
(defvar *act-ladder* \'(relu gelu swish mish))

(defun activation-successor (act)
  (let ((pos (position act *act-ladder*)))
    (if (and pos (< pos (1- (length *act-ladder*))))
        (nth (1+ pos) *act-ladder*)
        \'mish)))

;; Growth policy
(defun should-grow-p (current-loss prev-loss epoch)
  (and (> epoch 10)
       (> current-loss 0.15)
       (< (abs (- current-loss prev-loss)) 0.002)))

(defvar *prev-loss*    1.0)
(defvar *growth-count* 0)
(defvar *max-growths*  5)

;; Training loop with live self-modification
(with-training (net    mnist-net)
               (data   *train* :batch 64 :shuffle t)
               (loss   cross-entropy)
```

```
            (opt    (schedule (adam 0.001) :warmup 200 :decay cosine))
            (epochs 200)

  ;; 1. Vanishing gradient repair — every 5 epochs
  (when (zero-mod epoch 5)
    (let ((grad-norms (layer-gradient-norms mnist-net)))
      (loop for idx from 0
            for norm in grad-norms
            when (< norm 1e-5)
            do (let* ((cur  (layer-activation mnist-net idx))
                      (next (activation-successor cur)))
                 (format t "~&Epoch ~d: Layer ~d dying (~a → ~a)~%"
                         epoch idx cur next)
                 (rewrite mnist-net :layer idx :pos 2 next)))))

  ;; 2. Depth growth under stagnation
  (when (and (< *growth-count* *max-growths*)
             (should-grow-p current-loss *prev-loss* epoch))
    (let ((w (min 512 (* 2 (width-of mnist-net :last-hidden)))))
      (format t "~&Epoch ~d: Growing → width ~d~%" epoch w)
      (rewrite mnist-net :insert-before :output
        (dense w relu :drop 0.15))
      (incf *growth-count*)))

  ;; 3. Dropout annealing — every 25 epochs
  (when (zero-mod epoch 25)
    (let ((new-drop (max 0.05
                         (* 0.85 (dropout-of mnist-net :layer 1)))))
      (rewrite mnist-net :layer 1 :key :drop new-drop)))

  ;; 4. Logging — every 10 epochs
```

```
    (when (zero-mod epoch 10)
      (format t "~&Epoch ~d | Loss: ~,4f | Val: ~,2f% | Layers: ~d~%"
              epoch current-loss
              (* 100 (evaluate mnist-net *val* :metric :accuracy))
              (layer-count mnist-net)))

    ;; 5. Early stopping at target accuracy
    (when (> (evaluate mnist-net *val* :metric :accuracy) 0.993)
      (format t "~&Target reached at epoch ~d.~%" epoch)
      (signal \'stop-training))

    (setf *prev-loss* current-loss))

  ;; Results
  (format t "~&~%Growth events: ~d~%Modification history:~%" *growth-count*)
  (dolist (entry (history mnist-net))
    (format t "  ~s~%" entry))
  (inspect mnist-net)

  ;; Save the evolved architecture as readable Lisp source
  (with-open-file (out "mnist-evolved.lisp"
                       :direction :output :if-exists :supersede)
    (write (inspect mnist-net) :stream out))
  (format t "~&Evolved architecture saved → mnist-evolved.lisp~%")
```

The network starts with two layers. It may end anywhere from three to seven, with widths ranging from 64 to 512, activation functions upgraded from `relu` toward `gelu` or `swish` in any layer showing vanishing gradients, and dropout rates annealing from 0.15 toward 0.05. The final architecture is not specified by the programmer. It is discovered.

APPENDIX B

Formal Grammar of NeuralLISP Core Forms

The following EBNF sketch covers the seven primitive forms, their principal companion and derived forms, and the new `defsymbolic` and `with-rewrite-sandbox` forms introduced in this revision.

```
;; TOP-LEVEL FORMS
program       ::= { top-form }
top-form      ::= defactivation | defweights | defloss | defnet
               |  defmodule | defagent | defsymbolic
               |  with-training | with-rewrite-sandbox | cl-form

;; LAYER FORMS
dense-form    ::= "(" "dense"  width activation [ kw-args ] ")"
conv-form     ::= "(" "conv"   filters activation kernel [ kw-args ] ")"
recur-form    ::= "(" "recur"  width cell-type [ kw-args ] ")"
attend-form   ::= "(" "attend" heads dim [ kw-args ] ")"
output-form   ::= "(" "output" width activation ")"

;; NETWORK DEFINITION
defnet        ::= "(" "defnet" symbol "(" { symbol } ")" net-body ")"
net-body      ::= thread-form | let-form | values-form | cl-expr
thread-form   ::= "(" "->" expr { thread-stage } ")"

;; MODULE AND AGENT
defmodule     ::= "(" "defmodule" symbol "(" symbol { symbol } ")" net-body ")"
defagent      ::= "(" "defagent" symbol kw-args ")"

;; SYMBOLIC RULES
```

```
defsymbolic   ::= "(" "defsymbolic" symbol mode
                    [ ":predicate" lambda ] [ ":penalty" lambda ]
                    [ ":invariant" lambda ] [ ":message" string ]
                    [ ":category" keyword ] ")"
mode          ::= ":soft" | ":hard"


;; REWRITE SANDBOX
with-rewrite-sandbox ::= "(" "with-rewrite-sandbox"
                           "(" "net" symbol ")"
                           { cl-form }
                           [ "(" ":validate" expr ")" ]
                           [ "(" ":approve" approval-mode ")" ]
                           [ "(" ":rate-limit" integer ":per" interval ")" ] ")"


;; TRAINING
with-training ::= "(" "with-training"
                    "(" "net"    symbol ")"
                    "(" "data"   expr kw-args ")"
                    "(" "loss"   loss-expr ")"
                    "(" "opt"    optimizer-expr ")"
                    "(" "epochs" integer ")"
                    { cl-form } ")"


;; TERMINALS
cell-type     ::= "lstm" | "gru" | "rnn"
pool-strategy ::= "max" | "avg" | "global"
mask-type     ::= ":causal" | ":padding" | nil
init-strategy ::= ":xavier" | ":he" | ":random" | ":zero" | ":frozen"
training-mode ::= ":supervised" | ":adversarial" | ":actor-critic"
approval-mode ::= ":human" | ":auto" | lambda-form
interval      ::= ":epoch" | ":episode" | ":second"
```

The grammar is intentionally minimal. Any expression producing a tensor is a valid `cl-expr` in `net-body` and a valid `thread-stage`. The uniformity of the tensor type is what makes the grammar this concise.